Computers & Security 146 (2024) 104049

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

PHP-based malicious webshell detection based on abstract syntax tree simplification and explicit duration recurrent networks

Bailin Xie [a,*], Qi Li [a], Yu Wang [b]

[a] School of Information Science and Technology, Guangdong University of Foreign Studies, 510006, China
[b] Institute of Artificial Intelligence, Guangzhou University, 510006, China

ARTICLE INFO

Keywords:
Malicious webshells
Abstract syntax tree
Explicit duration recurrent network
Opcode sequences
Recurrent neural network

ABSTRACT

Malicious webshells are the most common attack scripts used by attackers in web penetration. Attackers typically obfuscate strings of PHP-based malicious webshells and encrypt communication traffic to bypass security devices. In this case, the opcode sequences of the PHP-based malicious webshells become excessively long and contain many irrelevant features, which affect the efficacy of the detection method. This study proposes a new PHP-based malicious webshell detection method. The proposed method introduces three simplification strategies for the three main types of nodes in the abstract syntax trees of PHP scripts to reduce the length and noise of opcode sequences of PHP-based malicious webshells. An explicit duration recurrent network (EDRN), a recurrent neural network based on an extended hidden semi-Markov model, is used to detect malicious webshells. Word2vec is adopted to convert the opcode sequences of the PHP scripts into vectors that serve as the input for the EDRN. Experiments were conducted using public datasets collected from GitHub. The experimental results indicated that EDRN outperformed popular recurrent neural networks. The proposed method demonstrated superior performance compared with several state-of-the-art approaches and mainstream tools, achieving an accuracy of 0.993, an F1 score of 0.990, and a recall rate of 0.991. When only 20% of the datasets were used for training, the proposed method achieved accuracy, recall, and F1 scores of 0.985, 0.983, and 0.980, respectively, significantly outperforming existing approaches.

1. Introduction

The Internet has pervaded almost all aspects of people's lives and profoundly impacted the development of the global economy and society. As one of the most popular Internet applications, the Web plays a very important role in people's social and economic activities. The Web has significantly facilitated people's daily lives and work; however, it has also become a major target for cyberattacks. After attackers compromise a web server, they usually exploit its vulnerabilities to upload a malicious webshell script (a script file written in PHP, ASP, or JSP) with specific functions and then access this script to remotely execute commands on the server to achieve information collection and data theft.

Owing to the characteristics of malicious webshells, such as strong concealment and ease of operation, they have become the most common attack scripts used by attackers for web penetration (V.Le et al., 2023; Zhang et al., 2020; Zhou and Zafarani, 2024). According to a Microsoft research report, Microsoft detected an average of 140,000 active malicious webshells per month between August 2020 and January 2021. Therefore, it is important to accurately and effectively identify malicious webshells stored on web servers.

Recently, PHP-based malicious webshell detection has garnered considerable attention from researchers (Hannousse et al., 2023; Suwais et al., 2023; Cheng et al., 2022). PHP is a dynamic scripting language executed in the Zend virtual machine by reading the script string, generating an abstract syntax tree and an opcode sequence, and executing each opcode. Most existing studies are based on the features of opcode sequences and use machine-learning or deep-learning algorithms to detect PHP-based malicious webshells. These methods directly extract features from opcode sequences (Shang et al., 2024).

Attackers typically obfuscate strings of PHP-based malicious webshells and encrypt communication traffic to bypass traditional security devices (Pang et al., 2023). In this case, the opcode sequences of the PHP-based malicious webshells become too long and contain many irrelevant features, which affect the efficacy of the detection method. To the best of our knowledge, no previous study has investigated this issue.

* Corresponding author.
E-mail address: bailinxie@gdufs.edu.cn (B. Xie).

https://doi.org/10.1016/j.cose.2024.104049
Received 8 June 2024; Received in revised form 3 August 2024; Accepted 6 August 2024
Available online 10 August 2024
0167-4048/© 2024 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

To address the issue of excessively long opcode sequences of PHP-based malicious webshells, this study proposes an abstract syntax tree simplification method called ASTSM. Considering the excellent performance of recurrent neural networks (RNNs) (Lipton and Berkowitz, 2015) in sequence analysis, we propose a PHP-based malicious webshell detection model based on an explicit duration recurrent network (EDRN) (Yu, 2022). In this model, we use Word2vec to convert the opcode sequences processed by the ASTSM, followed by an EDRN to accurately identify PHP-based malicious webshells. As a new recurrent neural network, the EDRN can capture the long-distance dependencies of input sequences more effectively than traditional RNNs.

The main contributions of this study are summarized as follows:

- For the first time, this study proposes an abstract syntax tree simplification method called ASTSM. The method involves different simplification strategies for the three main types of nodes in the abstract syntax tree of a PHP script, thereby reducing the length and noise of the opcode sequences of PHP-based malicious webshells.
- This study proposes a PHP-based malicious webshell detection model based on an EDRN. The model uses Word2vec to convert the ASTSM-processed opcode sequences into vectors as input and then uses EDRN to detect PHP-based malicious webshells.
- Based on experimental results from public datasets collected from GitHub, the performance of the EDRN was found to be better than that of other RNNs, and our malicious webshell detection method was superior to existing approaches and mainstream tools.

The remainder of this paper is organized as follows: Section 2 reviews recent studies on PHP-based malicious webshell detection. In Section 3, we describe a PHP-based malicious webshell detection method based on ASTSM and EDRN. Section 4 presents and discusses the evaluation and experimental results. Finally, Section 5 concludes the study.

2. Related works

In the field of PHP-based malicious webshell detection, Fang et al. (2018) used features, such as opcode sequences, and then adopted fastText and classification algorithms to detect malicious webshells. To detect malicious webshells more effectively, Zhou et al. (2021) took the opcode sequences as input and then used regularized neighborhood component analysis to reduce data dimensionality. Based on the features of opcode sequences and PHP source files, Cui et al. (2018) used gradient-boosting decision trees and random forest algorithms to identify malicious webshells. Nguyen et al. (2019) used Yara rules to build datasets, converted PHP source codes to opcode sequences, and finally adopted convolutional neural networks to detect malicious webshells. Liu et al. (2022) used an attention mechanism and a bidirectional gated recurrent unit (GRU) to detect malicious webshells. Tian et al. (2017) used Word2vec to convert samples into vectors and subsequently used a convolutional neural network to identify malicious webshells. Pan et al. (2021) used executable data and static text features of PHP scripts to detect malicious webshells. Li et al. (2019) used semantic and syntactic features to detect PHP-based malicious webshells. Zhao et al. (2021) analyzed the taint of opcode sequences of PHP scripts to identify malicious webshells.

In addition, Hannousse and Yahiouche (2021) used deep learning to identify malicious webshells written in PHP, ASP, JSP or ASPX. Zhang et al. (2018) used long short-term memory (LSTM) networks and convolutional neural networks to detect malicious webshell traffic. Feng et al. (2024) used a graph neural network to identify malicious webshells on an industrial Internet.

Most existing approaches extract features directly from the opcode sequences. However, owing to confusion and encryption, the opcode sequence of a PHP-based malicious webshell is often excessively long and contains many irrelevant features. The existing methods do not address these issues. The twofold aims of this study are (i) to detect PHP-based malicious webshells more effectively and (ii) to introduce simplification strategies for the abstract syntax tree to reduce the length and noise of the opcode sequence of the PHP-based malicious webshell. Subsequently, we propose a new PHP-based malicious webshell detection model based on EDRN.

3. PHP-based malicious webshell detection method

The framework of our PHP-based malicious webshell detection method is illustrated in Fig. 1. In this method, PHP-Parser (2024) is used to parse PHP scripts into abstract syntax trees, and then ASTSM is used to simplify the abstract syntax trees. Subsequently, VLD (2022) is adopted to extract the opcode sequences, which are then converted into vectors using Word2vec. Finally, an EDRN is used to detect PHP-based malicious webshells. Fig. 2 shows the opcode sequence of a PHP-based malicious webshell, which is {BEGIN_SILENCE, FETCH_R, FETCH_DIM_R, INCLUDE_OR_EVAL, END_SILENCE, RETURN} (Xie and Li, 2022).

3.1. ASTSM

A flowchart of the proposed ASTSM is shown in Fig. 3. The nodes in the abstract syntax tree of the PHP script primarily include statement, expression, and scalar nodes. ASTSM adopts different simplification strategies for the three main types of nodes in an abstract syntax tree.

The pseudocode of ASTSM is shown in Algorithm 1. The traversal of each node in the abstract syntax tree is divided into two phases: entering and leaving the node. Before leaving the current node, ASTSM calls the corresponding simplification function to prune the node. In Algorithm 1, "$handle_Statement()$" denotes the statement node simplification function, "$handle_Expression()$" denotes the expression node simplification function, "$handle_Scalar()$" denotes the scalar node simplification function, and "$handle_Other()$" denotes the simplification function of other nodes. $\boldsymbol{Q}$, $\boldsymbol{R}$, and $\boldsymbol{Z}$ denote arrays that are defined as follows:

$\boldsymbol{Q}$: Used to store variables that have been assigned before entering the current node. If the current node uses a variable in $\boldsymbol{Q}$, we will use the value of the variable in $\boldsymbol{Q}$ to replace the variable. In addition, we set a count flag to record whether the variable in $\boldsymbol{Q}$ has been used or not.

$\boldsymbol{R}$: Used to store custom functions in the PHP script. If the current node calls a non-built-in function, ASTSM obtains the corresponding function by searching for $\boldsymbol{R}$. In addition, we set a count flag to record whether a function in $\boldsymbol{R}$ has been called or not.

$\boldsymbol{Z}$: Used to store functions where the number of input parameters during function calls does not match the preset number of parameters.

In addition, the initial list of functions is defined as "f_list". If the assignment of a variable is based on the result of the execution of a built-in function in PHP, we first determine whether the name of the calling function is in "f_list". To avoid dealing with potentially risky functions, we only include encoding functions, encryption and decryption functions, and some string processing functions, such as "$base64_encode$", "$base64_decode$", "$urlencode$", "$urldecode$", "$md5$", "chr", "ord", "$substr$", "$strrev$", "str_rot13", "$gninflate$", "$gzdeflate$", "$gzcompress$", "$gzuncompress$", "$trim$", "$implode$", "$str_replace$", and "$substr_replace$" in "f_list".

3.1.1. Statement node simplification strategy

The pseudocode of "$handle_Statement()$" is presented in Algorithm 2. The child nodes of the statement nodes include the conditional statement child nodes and functional definition statement child nodes. The child nodes of conditional statement child nodes include "If", "ElseIf", and "Else".

For conditional statement child nodes, the attacker can enter arbitrary commands through the PHP-based malicious webshell to obtain

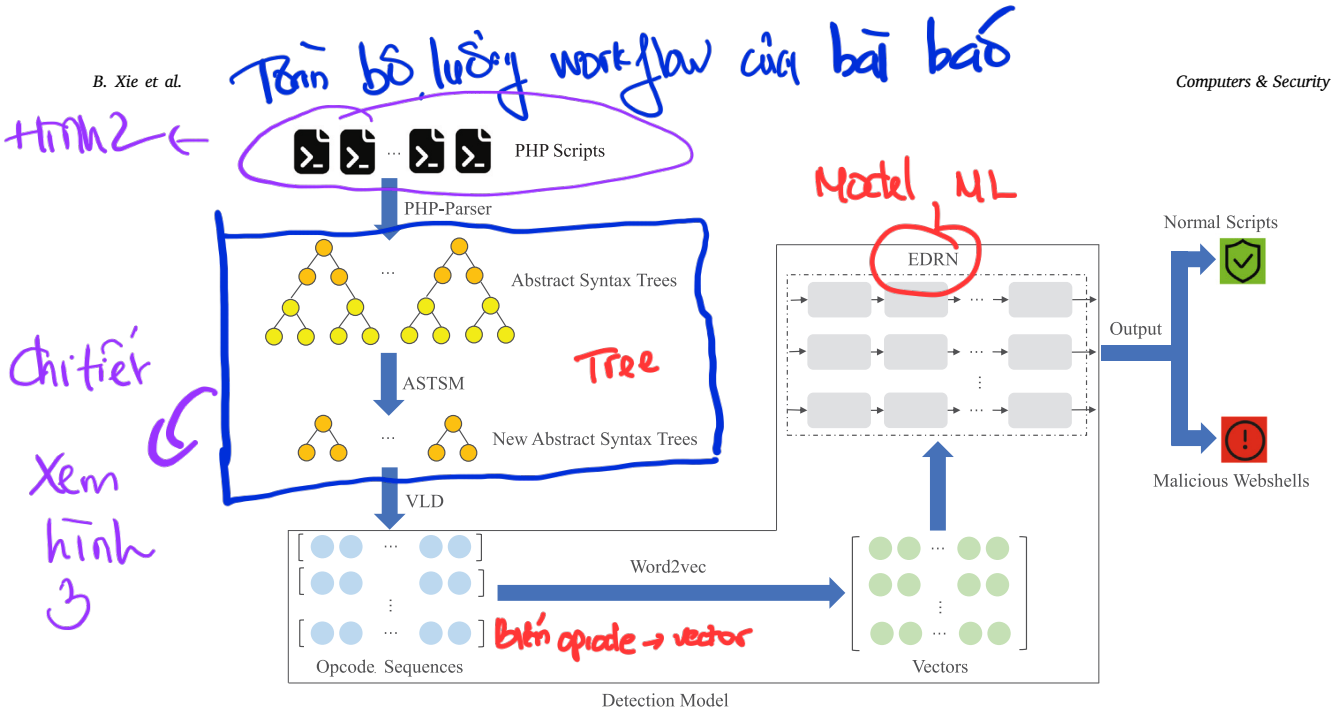


Fig. 1. PHP-based malicious webshell detection framework.

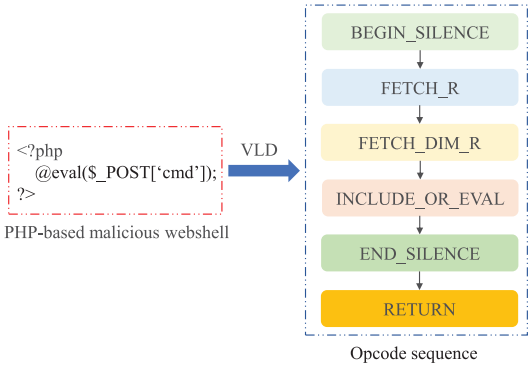


Fig. 2. PHP-based malicious webshell and its opcode sequence.

Algorithm 1 ASTSM

Input: initial abstract syntax tree *AST*, *f_list*
Output: new abstract syntax tree *new_AST*
1: *node* in *AST*
2: **while** leaving the current node **do**
3: **if** *node* ∈ Statement **then**
4: *new_node* ← *handle_Statement*(*node*,***Q***, ***R***,***Z***,*f_list*)
5: **else if** *node* ∈ Expression **then**
6: *new_node* ← *handle_Expression*(*node*,***Q***, ***R***,***Z***,*f_list*)
7: **else if** *node* ∈ Scalar **then**
8: *new_node* ← *handle_Scalar*(*node*,***Q***, ***R***,***Z***,*f_list*)
9: **else**
10: *new_node* ← *handle_Other*(*node*,***Q***, ***R***,***Z***,*f_list*)
11: **end if**
12: **end while**
13: *new_AST* ← *update*(*node*,*new_node*)
14: **return** *new_AST*

information on the web server; therefore, we do not need to consider the attacker's input but only need to consider whether there are statements such as "_GET" and "_POST" to obtain external input after the conditional statement.

Attackers often use custom functions to escape the detection of PHP-based malicious webshells; therefore, when a custom function exists in the function definition statement child node, we first use ***R*** to store the custom function and then remove the entire node from the abstract syntax tree. If the custom function is called later, it is removed from ***R*** and executed conditionally.

Algorithm 2 *handle_Statement*()

Input: initial node *node*, ***Q***, ***R***,***Z***,*f_list*
Output: new node *new_node*
1: **if** *node* ∈ Statement/If or Statement/ElseIf or Statement/Else **then**
2: *controllable_input*={_GET, _POST,_REQUEST,_COOKIE,_SESSION,_SERVER,_ENV }
3: **end if**
4: **if** *node* → *expr* ∈ Statement/Function and *node* → *name* → *name* in ***Q*** **then**
5: ***R***[*node* → *name* → *name*]=*node*
6: **end if**
7: *new_node* ← *remove_node*(*node*)
8: **return** *new_node*

3.1.2. Expression node simplification strategy

The pseudocode of "*handle_Expression*()" is presented in Algorithm 3. We mainly simplify the child nodes of "Variable", "ConstFetch", "ArrayDimFetch", "Assign", "AssignOp", "BinaryOp", and "Array_" under the expression nodes. The structural information of the child nodes is listed in Table 1. Our strategy for child nodes of "Variable", "ConstFetch", "ArrayDimFetch", and "Array_" is to find the corresponding value of the variable in ***Q*** and then replace the variable with the corresponding value.

The strategy for other child nodes under the expression nodes is as follows:

(1) Assign: If the right operand is constant, we place this constant into ***Q*** and remove its right operand from the abstract syntax tree; if it is a variable that exists in ***Q***, we replace the variable with its corresponding value from ***Q***.

(2) AssignOp: If the right operand exists in ***Q***, we take the value of the variable from ***Q*** to perform the corresponding operation. Subsequently, we update the value of the variable in ***Q*** and remove the right operand from the abstract syntax tree.

(3) BinaryOp: If its left or right operand is a variable that exists in ***Q***, we replace it with the corresponding value from ***Q***.

3.1.3. Scalar node simplification strategy

A scalar node represents a specific scalar value that can be a number, string, or Boolean value. Our simplification strategy for scalar

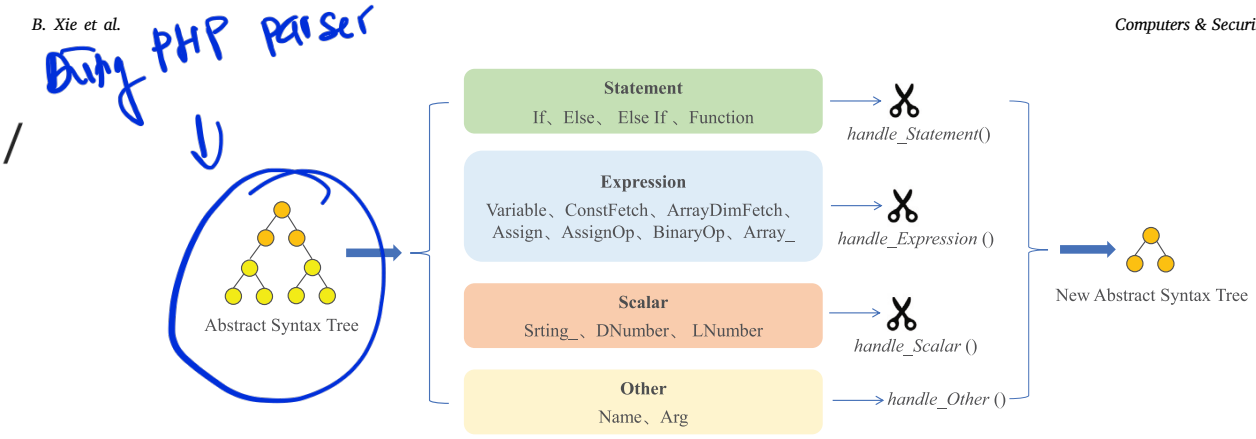


Fig. 3. Flowchart of ASTSM.

Algorithm 3 *handle_Expression*()

Input: initial node *node*, ***Q***, ***R***,***Z***,*f_list*
Output: new node *new_node*
1: **if** *node* ∈ Expression/Assign and *node* → *var* ∈ Expression/Variable and *node* → *expr* ∈ Expression/Array_ and *node* → *var* → *name* in ***Q*** **then**
2: ***Q***[*node* → *var* → *name*]= *node* → *expr* → *items*
3: *new_node* ← *remove_node*(*node*)
4: **end if**
5: **if** *node* ∈ Expression/AssignOp and *node* → *var* ∈ Expression/Variable and *node* → *expr* ∈ Expression/Variable and *node* → *expr* → *name* in ***Q*** **then**
6: *node* → *expr* ← *new_node*(***Q***[*node* → *expr* → *name*])
7: **end if**
8: **if** *node* ∈ Expression/BinaryOp and *node* → *left* ∈ Expression/Variable and *node* → *left* → *name* in ***Q*** **then**
9: *node* → *left* ← *new_node*(***Q***[*node* → *left* → *name*])
10: **end if**
11: **if** *node* ∈ Expression/BinaryOp and *node* → *right* ∈ Expression/Variable and *node* → *right* → *name* in ***Q*** **then**
12: *node* → *right* ← *new_node*(***Q***[*node* → *right* → *name*])
13: **end if**
14: **return** *new_node*

Table 1
Structure of child nodes under the expression nodes.

Child nodes	Structural information
Assign	var(variable),expr(right operand),operator
Variable	name
ArrayDimFetch	var(variable),dim(index)
ConstFetch	name
Array_	items, attributes
BinaryOp	left(left operand),right(right operand),operator
AssignOp	var (variable),expr(right operand),operator

nodes is as follows: when entering the scalar node, the value of the scalar node is stored; when leaving the scalar node, the value of the scalar node is replaced with the corresponding reference position; and the results generated by function calls are converted and updated to the node. Finally, unreferenced scalar nodes are removed from the abstract syntax tree.

The time consumption of Algorithm 1, Algorithm 2, and Algorithm 3 is primarily the query of arrays ***Q***, ***R***, and ***Z***; therefore, their time complexity is approximately $max\{O(len(\boldsymbol{Q})), O(len(\boldsymbol{R})), O(len(\boldsymbol{Z}))\}$, where *len*() represents the function used to obtain the length of the array. Therefore, the ASTSM proposed in this study has high speed.

3.2. Detection model

A flowchart of the proposed detection model is shown in Fig. 1. The details of each part of the detection model are as follows:

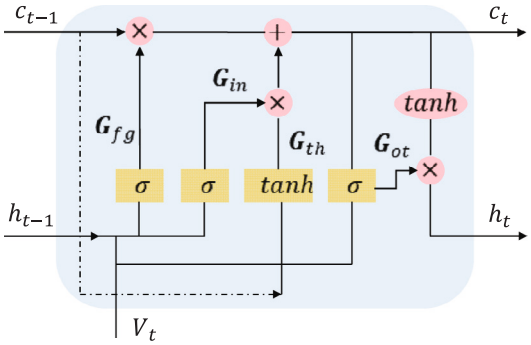


Fig. 4. Unit structure of EDRN.

3.2.1. Opcode sequences

Let the dataset contain T opcode sequences, namely, $U = \{O_1, O_2, ..., O_T\}$,where O_t=[$w_{(t,1)}$, $w_{(t,2)}$, $w_{(t,3)}$, …, $w_{(t,L_t)}$] is the tth ($1 \le t \le T$) opcode sequence, $w_{(t,l)}$ denotes the lth opcode of the tth opcode sequence, and L_t is the length of the tth opcode sequence.

3.2.2. Vectors

We adopt Word2vec to convert the opcode sequences into vectors, which are then used as inputs to the recurrent neural network. Let V_t=[$v_{(t,1)}$, $v_{(t,2)}$, $v_{(t,3)}$, …, $v_{(t,L_t)}$] denote the vector of the tth opcode sequence. In this study, we set the dimensions of the vector to 100 to prevent the vector matrix from becoming overly sparse.

3.2.3. EDRN

An EDRN is a recurrent neural network based on an extended hidden semi-Markov model (HSMM). Therefore, the EDRN can learn the hidden states with any duration distribution (Yu, 2022). We adopted EDRN to construct a PHP-based malicious webshell detection model.

The unfolded EDRN unit is shown in Fig. 4, where c_t denotes all possible state probabilities at the tth time, and h_t denotes the next possible states' predicted probabilities at the $(t+1)^{th}$ time. In Fig. 4, $\boldsymbol{G}_{in}$, $\boldsymbol{G}_{fg}$, $\boldsymbol{G}_{ot}$, and $\boldsymbol{G}_{th}$ represent the input, forget, output, and tanh gates, respectively.

Let EDRN contain N macrostates, and each macrostate contains M substates. Let $s_t = (n, m)$ represent the substate m of state n at the tth time: $1 \le n \le N, 1 \le m \le M$. Let $a_{(n,m),(p,q)}$ represent the probability of an interstate transition from $s_t = (n, m)$ to $s_{t+1} = (p, q)$. It is defined by Eq. (1): $1 \le p \le N, 1 \le q \le M$.

$$a_{(n,m),(p,q)} \equiv Pr\left[s_t = (n,m), s_{t+1} = (p,q)\right] \tag{1}$$

The $\boldsymbol{G}_{in}$, $\boldsymbol{G}_{fg}$, $\boldsymbol{G}_{ot}$, and $\boldsymbol{G}_{th}$ are expressed as follows:

$$\boldsymbol{G}_{fg} = \sigma\left(\boldsymbol{A}_{fg} h_{t-1} + \boldsymbol{B}_{fg} V_t + b_{fg}\right) \tag{2}$$

$$\boldsymbol{G}_{in} = \sigma\left(\boldsymbol{A}_{in} h_{t-1} + \boldsymbol{B}_{in} V_t + b_{in}\right) \tag{3}$$

$$\boldsymbol{G}_{th} = tanh\left(\boldsymbol{A}_{pt} c_{t-1} + \boldsymbol{A}_{th} c_{t-1}(:, M) + \boldsymbol{B}_{th} V_t + b_{th}\right) \tag{4}$$

$$\boldsymbol{G}_{ot} = \sigma\left(\boldsymbol{A}_{ot} c_t(:, M) + \boldsymbol{B}_{ot} V_t + b_{ot}\right) \tag{5}$$

In the previous equations, σ represents the sigmoid function, *tanh* represents the hyperbolic tangent function, ":" represents all the states, and $c_t = \{c_t(p,q)\}$ can be calculated by Eq. (6), as follows:

$$c_t = c_{t-1} * \boldsymbol{G}_{fg} + \boldsymbol{G}_{th} * \boldsymbol{G}_{in} \tag{6}$$

In Eq. (6), "∗" represents the Python operator of elementwise production.

In Eqs. (2) and (3), $h_{t-1} = \{h_{t-1}(p)\}$ and $h_{t-1}(p)$ are expressed by Eq. (7), where $c^{**}_{t-1}(p,m)$ is calculated using Eq. (8).

$$h_{t-1}(p) = \sum_{m<M} c^{**}_{t-1}(p,m) + \sum_{n} c^{**}_{t-1}(n,M)\, a_{(n,M),p} \tag{7}$$

$$c^{**}_{t-1}(p,m) = c_{t-1}(p,m)\, \beta_{(p,m)} \tag{8}$$

In Eq. (8), $\beta_{(p,m)}$ is the probability that state (p,m) will not jump to the absorbing state.

Let $c^{**}_t = \left\{c^{**}_t(p,q)\right\}$; it can be calculated using Eq. (9).

$$c^{**}_t = tanh\left(c_t\right) * \boldsymbol{G}_{ot} \tag{9}$$

In the above formulas, $\boldsymbol{A}_{fg}$, $\boldsymbol{A}_{in}$, $\boldsymbol{A}_{ot}$, and $\boldsymbol{A}_{th}$ are similar to the probability matrix of the state transition of HSMM; $\boldsymbol{A}_{pt}$ is the substate transition probability matrix; $\boldsymbol{B}_{fg}$, $\boldsymbol{B}_{in}$, $\boldsymbol{B}_{ot}$, and $\boldsymbol{B}_{th}$ are similar to the probability matrix of the observation of HSMM; and b_{fg}, b_{in}, b_{ot}, and b_{th} are bias parameters that obey a marginal distribution (Xie et al., 2023).

Finally, h_t is updated, and (c_t, h_t) is returned to the next unit. $h_t(:)$ is expressed by Eq. (10), where $\boldsymbol{A}_{st}$ is the state prediction probability matrix.

$$h_t(:) = \sum_{m=1}^{M-1} c^{**}_t(:,m) + c^{**}_t(:,M)\, \boldsymbol{A}_{st} \tag{10}$$

4. Experiments

4.1. Datasets

To evaluate our method, we collected 2,229 PHP-based malicious webshell samples from GitHub, including Xl7dev (2017), Ysrc (2018), JohnTroony (2020), and BlackArch (2021). In addition, we used three open-source content management system (CMS) projects as normal samples: Craftcms (2022), WordPress (2022), and Yii2 (2022). Details of the datasets are specified in Table 2.

The frequency rankings of opcodes of normal and malicious webshell samples are listed in Table 3. As can be observed from Table 3, the top 10 most frequent opcodes of the normal samples and the malicious webshell samples are almost different; therefore, it is reasonable to identify malicious webshells based on opcode sequences.

4.2. Experimental results and analysis

The experimental environment was built using Ubuntu (version 20.04), Keras (version 2.3.1), TensorFlow (version 1.15.0), Python (version 3.6), and Numpy (version 1.19.2).

Table 2
Datasets.

Datasets	Sources	Number of samples	Years
Malicious webshell	JohnTroony	118	2020
	Xl7dev	281	2017
	Ysrc	1,637	2018
	BlackArch	193	2021
CMS	Cratfcms	1,191	2022
	WordPress	1,160	2022
	Yii2	996	2022

Table 3
Top 10 most frequent opcodes.

Frequency rankings	Opcodes of normal samples	Opcodes of malicious webshell samples
1	DO_FCALL	CONCAT
2	SEND_VAL_EX	INIT_FCALL
3	ASSIGN	SEND_VAL
4	INIT_METHOD_CALL	DO_ICALL
5	RETURN	ASSIGN
6	SEND_VAR_EX	FETCH_DIM_R
7	INIT_FCALL_BY_NAME	JMPZ
8	JMPZ	DO_FCALL
9	FETCH_OBJ_R	SEND_VAR
10	ADD_ARRAY_ELEMENT	ECHO

Table 4
Changes in length of malicious webshell opcode sequences.

Length intervals	Number (Before simplification)	Number (After simplification by ASTSM)
0–100	965	1,067
100–200	157	104
200–300	79	79
300–400	55	71
400–500	22	48
500–600	18	20
600–700	35	24
700–800	15	26
800–900	15	9
900–1000	16	10

4.2.1. ASTSM experiments

In the malicious webshell dataset, 75% of the opcode sequences are between 0 and 1,000 in length. After simplification by ASTSM, the lengths of 1,479 malicious webshell opcode sequences changed, with more than half of them reduced by more than 10%. Table 4 lists the changes in the lengths of the malicious webshell opcode sequences with lengths less than 1,000. After simplification using ASTSM, the number of malicious webshell opcode sequences that were less than 100 in length increased by 102. This indicates that ASTSM can effectively reduce the length of malicious webshell opcode sequences.

4.2.2. EDRN parameters

(1)Parameters N and M

Two parameters must be set for EDRN: N (similar to the parameter *units* in traditional RNNs) and M. We explored the impact of different parameter values on the performance of EDRN. In the test, we randomly divided the dataset into a training, test, and validation sets in a ratio of 6:2:2.

(i) Evaluation indicators

In this experiment, we used accuracy to evaluate the classification capability of the model, recall rate to evaluate the model's capability to identify malicious webshells, F1 score to evaluate the robustness of the model, and loss value to evaluate the degree of difference between the predicted values of the model and true values. Based on the confusion matrix shown in Fig. 5, the formulas for the accuracy, recall, and F1 scores are as follows:

		Predicted values: Malicious webshell opcode sequences	Predicted values: Normal opcode sequences
Actual values	Malicious webshell opcode sequences	True positive (TP)	False negative (FN)
	Normal opcode sequences	False positive (FP)	True negative (TN)

Fig. 5. Confusion matrix.

$$accuracy = \frac{TP + TN}{TP + FN + FP + TN} \tag{11}$$

$$recall = \frac{TP}{TP + FN} \tag{12}$$

$$F1 = 2 \times \left(\frac{\frac{TP}{TP+FP} \times recall}{\frac{TP}{TP+FP} + recall} \right) \tag{13}$$

(ii) Experiments based on datasets not processed by ASTSM

We used datasets that were not processed by ASTSM for the experiments, and the results are shown in Fig. 6. As shown in Fig. 6, EDRN performs best when M is set to 6 and N is set to 35, and its confusion matrix is shown in Fig. 7. In addition, we analyzed the distribution of the hidden states of the EDRN. Fig. 8 presents a box plot of the hidden state distribution of normal opcode sequences, and Fig. 9 presents a box plot of the hidden state distribution of malicious webshell opcode sequences. It can be observed that the hidden state distribution of the normal opcode sequences was different from that of the malicious webshell opcode sequences.

(iii) Experiments based on datasets processed by ASTSM

We used the datasets processed by ASTSM for the experiments, and the results are shown in Fig. 10. The experimental results showed that the recall, accuracy, and F1 values of the EDRN improved, and the loss value decreased. When M is set to 6 and N is set to 35, EDRN still performs well, and its confusion matrix is shown in Fig. 11. As shown in Figs. 7 and 11, the performance of EDRN improved after the application of ASTSM. Therefore, in subsequent comparative experiments, we set M to 6 and N to 35. Figs. 12 and 13 show box plots of the hidden state distributions of the normal and malicious webshell opcode sequences, respectively. After ASTSM was applied, the difference in the distribution of hidden states between normal and malicious webshell opcode sequences became even greater.

(2) Parameter epoch

During recurrent neural network training, if the epoch value is set too small, the model will not learn sufficiently. If the epoch value is set too large, the training time of the model is excessively long. To explore the reasonable value of the epochs, we tested the performances of the EDRN and traditional RNNs, including SimpleRNN (Lipton and Berkowitz, 2015), LSTM (Hochreiter and Schmidhuber, 1997), PLSTM (Sak et al., 2014), and GRU (Cho et al., 2014), for different epochs. The experimental results are shown in Fig. 14. The parameter settings for each recurrent neural network are specified in Table 5.

As shown in Fig. 14, when the epoch was approximately 100, the change trends of the accuracy, recall, F1 score, and loss value of all the models were nearly flat, and all the models achieved a relatively stable and optimal performance. When the epoch was approximately 120, all indicators of the SimpleRNN fluctuated significantly. Therefore, in subsequent comparative experiments, we set the epoch to 100.

Table 5
Model parameter settings.

Models	M	N	$batch_size$	$units$	Total number of parameters
LSTM	–	–	32	128	117,248
PLSTM	–	–	32	128	117,632
GRU	–	–	32	128	87,936
EDRN	6	35	–	–	115,990

Table 6
Ablation results based on datasets not processed by ASTSM.

Input gate	Forget gate	Output gate	Tanh gate	Recall	Accuracy	F1	Loss
×	✓	✓	✓	0.828	0.835	0.821	0.259
✓	×	✓	✓	0.840	0.847	0.832	0.243
✓	✓	×	✓	0.775	0.786	0.764	0.338
✓	✓	✓	×	0.738	0.752	0.725	0.376

Table 7
Ablation results based on datasets processed by ASTSM.

Input gate	Forget gate	Output gate	Tanh gate	Recall	Accuracy	F1	Loss
×	✓	✓	✓	0.837	0.848	0.832	0.245
✓	×	✓	✓	0.853	0.861	0.849	0.226
✓	✓	×	✓	0.795	0.802	0.790	0.323
✓	✓	✓	×	0.762	0.777	0.754	0.351

4.2.3. Ablation experiment

To verify that there were no redundant gates in the EDRN unit, we removed one gate at a time from the EDRN unit and evaluated its performance. Table 6 presents the ablation results based on the datasets that were not processed by ASTSM, and Table 7 lists the ablation results based on the datasets that were processed by ASTSM. When any one of the gates in the EDRN unit was removed, the performance of the EDRN degraded significantly. Therefore, every gate in the EDRN unit was deemed important.

4.2.4. Comparative experiments with EDRN and traditional RNNs

(1) Comparative experiments based on datasets not processed by ASTSM

Based on the datasets not processed by ASTSM, we used different proportions of the training set to compare the EDRN with traditional RNNs. The experimental results are presented in Fig. 15.

As shown in Fig. 15, when 20% of the samples were used for training, the accuracy, F1 score, and loss value of the EDRN were only 0.001 worse than those of the GRU, whereas the recall rate of the EDRN was much higher than that of the GRU.

When 40% of the samples were used for training, the performance of each model improved significantly. The performance of GRU was the closest to that of EDRN; however, GRU was inferior to EDRN in terms of its malicious webshell identification capability.

When 60% of the samples were used for training, the performances of LSTM, PLSTM, and SimpleRNN were still significantly improved, but the performance improvements of EDRN and GRU were minimal. The loss values for all models were reduced significantly.

When 80% of the samples were used for training, the identification capability of each model for malicious webshells tended to be the same except for SimpleRNN. The EDRN had the highest recall, accuracy, and F1 score.

In summary, when the training dataset was small, EDRN exhibited the best malicious webshell identification capability. When the training dataset was sufficient, the performance of EDRN was not inferior to that of other RNNs. In the real world, malicious webshell samples are difficult to obtain; therefore, EDRN is more suitable for practical applications.

(2) Comparative experiments based on datasets processed by ASTSM

Based on the datasets processed by ASTSM, we used different proportions of the training set to compare the EDRN with traditional RNNs.

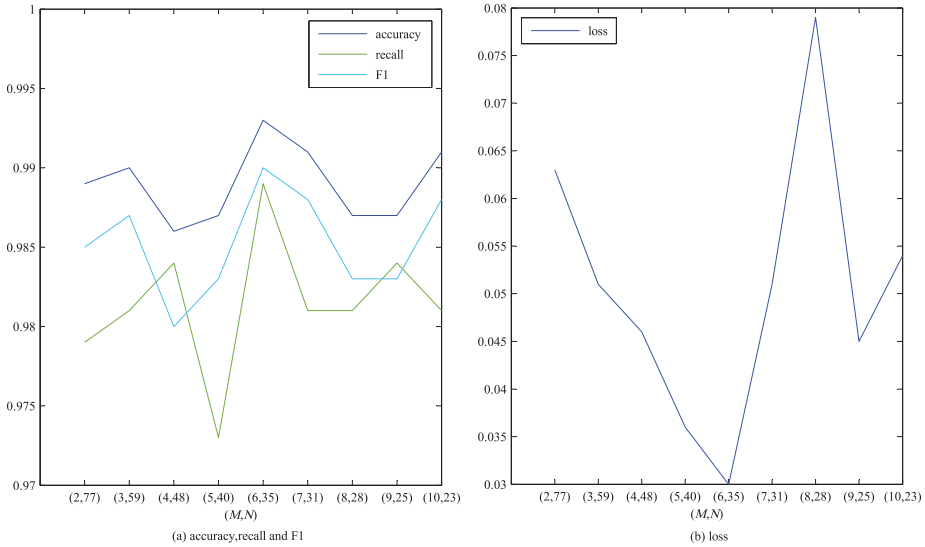


Fig. 6. Impact of different parameter values on EDRN based on datasets not processed by ASTSM.

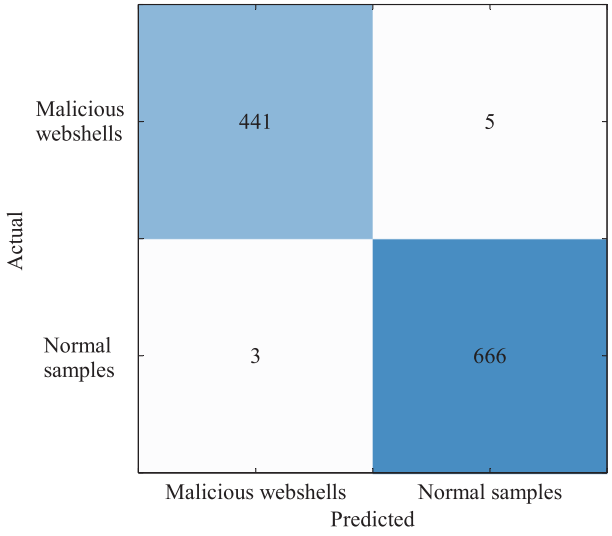


Fig. 7. Confusion matrix of EDRN based on datasets not processed by ASTSM.

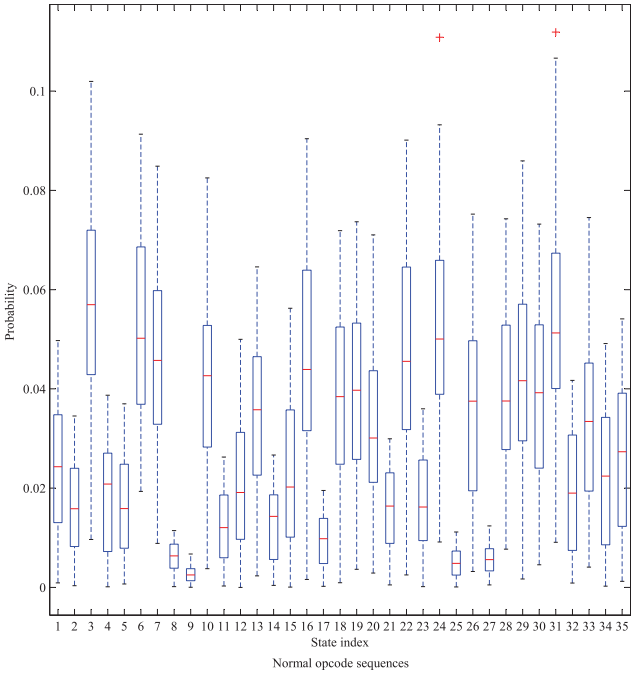


Fig. 8. Box plot of hidden state distribution of normal opcode sequences based on datasets not processed by ASTSM.

The experimental results are shown in Fig. 16. As can be seen from Figs. 15 and 16, the malicious webshell identification capability of each recurrent neural network significantly improved after the application of ASTSM.

Fig. 17 shows the performance changes of each recurrent neural network after ASTSM was applied. As shown in Fig. 17, ASTSM improves the evaluation indicators of each recurrent neural network, particularly the recall rate. The results show that ASTSM can effectively reduce the noise in malicious webshell opcode sequences and shorten the length of malicious webshell opcode sequences. Therefore, the features of malicious webshell opcode sequences can be further highlighted, and RNNs can capture these features more easily.

4.2.5. Comparative experiments with existing approaches

We tested the proposed method against existing approaches. In this experiment, recall, accuracy, and F1 score were used as evaluation metrics. The experimental results obtained when 60% of the samples were used for training are listed in Table 8.

Our method performed the best in terms of recall and F1 value, with an accuracy that was only 0.001 lower than that of Liu et al. (2022).

Table 8
Comparison results with existing approaches when 60% of the samples are used for training.

Methods	Years	Accuracy	Recall	F1
Fang et al. (2018)	2018	0.992	0.977	0.978
Zhou et al. (2021)	2021	0.992	0.933	0.933
Cui et al. (2018)	2018	0.991	0.990	0.990
Nguyen et al. (2019)	2019	0.990	0.984	0.972
Liu et al. (2022)	2022	0.994	0.986	0.984
Suwais et al. (2023)	2023	0.989	0.972	0.965
Our method	2024	0.993	0.991	0.990

In terms of the recall rate, our method was much higher than those of Zhou et al. (2021), Suwais et al. (2023), and Fang et al. (2018); slightly higher than those of Nguyen et al. (2019) and Liu et al. (2022);

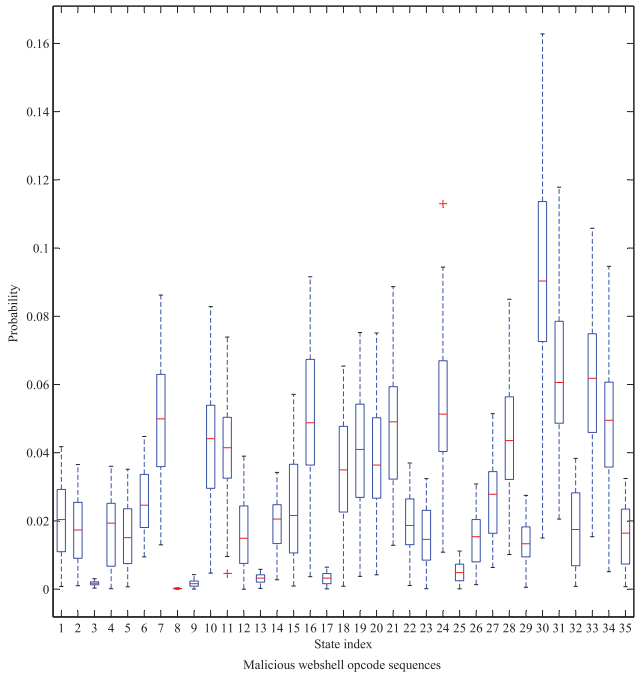


Fig. 9. Box plot of hidden state distribution of malicious webshell opcode sequences based on datasets not processed by ASTSM.

and 0.001 higher than those of Cui et al. (2018). Consequently, our method exhibited the best overall performance.

The experimental results obtained when only 20% of the samples were used for training are listed in Table 9. As presented in Table 9, our method was significantly superior to the other methods. Since it is difficult to obtain malicious webshell samples, our method is more suitable for real-world malicious webshell detection.

4.2.6. Comparative experiments with existing mainstream tools

We compared the proposed method with existing mainstream tools. The main tools used are as follows:

PHP Shell Detector (2016): This tool detects malicious webshells by building a signature library of malicious webshells. In comparative experiments, we considered the suspicious files and malicious webshells identified by the tool to be the malicious webshells detected by the tool.

Table 9
Comparison results with existing approaches when 20% of the samples are used for training.

Methods	Years	Accuracy	Recall	F1
Fang et al. (2018)	2018	0.946	0.933	0.930
Zhou et al. (2021)	2021	0.949	0.906	0.901
Cui et al. (2018)	2018	0.953	0.949	0.945
Nguyen et al. (2019)	2019	0.948	0.941	0.936
Liu et al. (2022)	2022	0.952	0.945	0.942
Suwais et al. (2023)	2023	0.934	0.927	0.921
Our method	2024	0.985	0.983	0.980

D shield (2023): This tool is widely used in China. In comparative experiments, the samples with a risk level of 1–5 identified by the tool were considered malicious webshells detected by the tool.

PHP Malware Finder (2022): This tool is a malicious webshell detection tool based on Yara rules and pattern matching techniques. In comparative experiments, the samples that met the rules defined by the tool were considered malicious webshells detected by the tool.

WebShellkiller (2021): There are two strategies for this tool, namely Strategy 1: "Accuracy Priority, Reduce False Positive Rate", and Strategy 2: "Number of Detections First". In comparative experiments, we tested the performance of the tool under two different strategies.

SHELLPUB (2022): This tool divides results into suspicious and malicious webshell samples. In the comparative experiments, suspicious and malicious webshell samples identified by the tool were considered malicious webshells detected by the tool.

WEBDIR+ (2021): This tool outputs results in the form of a JSON string, including the number and type of malicious webshells.

CloudWalker (2018): This tool outputs detection results by classifying them into risk levels. In the comparative experiments, files with a risk level of 1–5 were considered malicious webshells detected by the tool.

In this experiment, we used accuracy, detection rate (DR), false positive rate (FPR), F1 score and false negative rate (FNR) as evaluation indicators.

Based on the confusion matrix shown in Fig. 5, the DR, FPR, and FNR were calculated as follows:

$$DR = \frac{TP}{TP + FN} \tag{14}$$

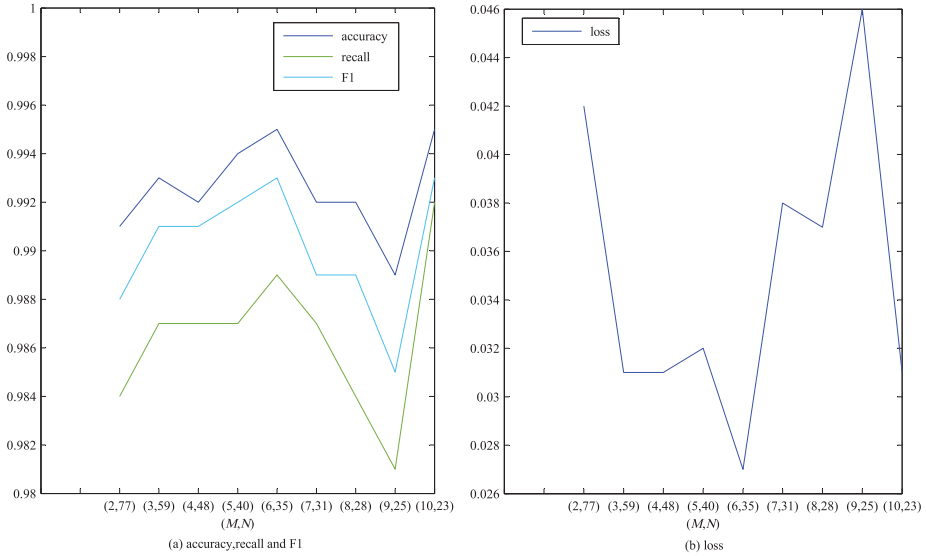


Fig. 10. Impact of different parameter values on EDRN based on datasets processed by ASTSM.

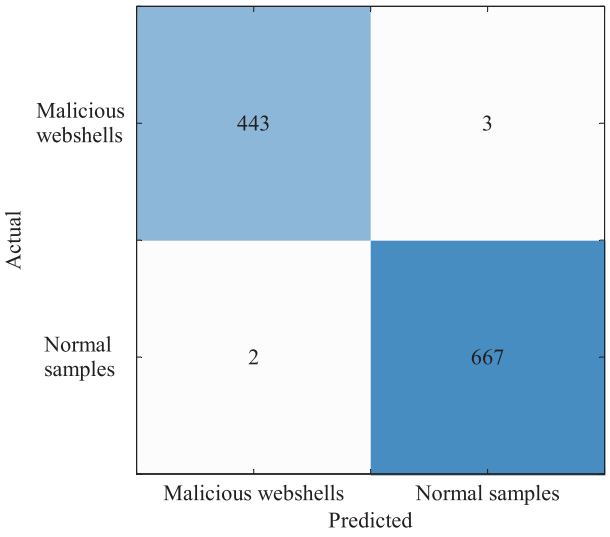


Fig. 11. Confusion matrix of EDRN based on datasets processed by ASTSM.

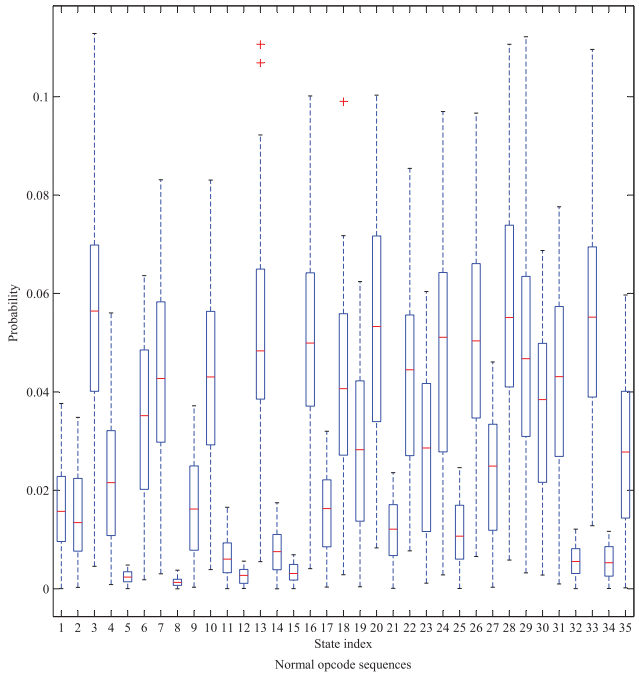


Fig. 12. Box plot of hidden state distribution of normal opcode sequences based on datasets processed by ASTSM.

$$FPR = \frac{FP}{TN + FP} \tag{15}$$

$$FNR = \frac{FN}{TP + FN} \tag{16}$$

Table 10 lists the obtained experimental results. Our method achieved the best performance, with an accuracy of 0.993, a DR of 0.991, and an FPR of 0.006. PHP Shell Detector (2016) has an FPR of 0.247, which is the highest of any tool because the tool's signature library has not been updated for a long time.

PHP Malware Finder (2022) detects malicious webshells based on rule matching and cannot accurately identify new types of malicious webshells. The signature library of D shield (2023) is frequently updated. Therefore, D shield (2023) can effectively detect existing malicious webshells and has become the most popular malicious webshell

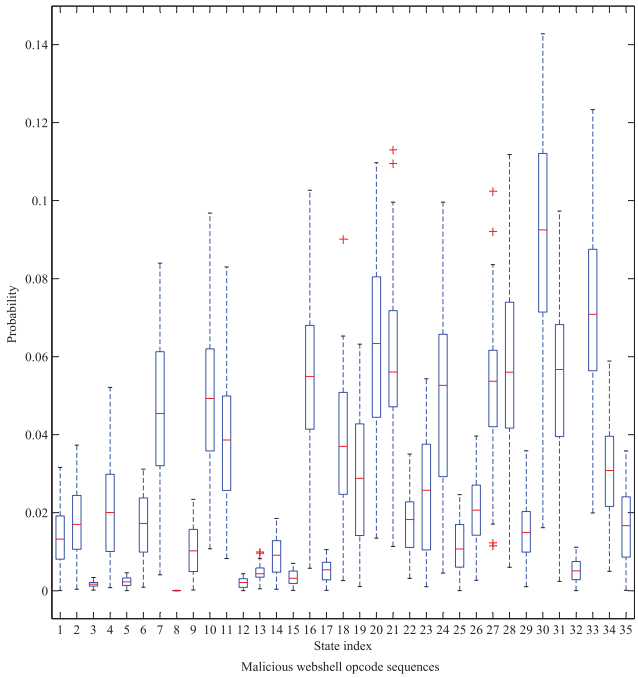


Fig. 13. Box plot of hidden state distribution of malicious webshell opcode sequences based on datasets processed by ASTSM.

detection tool in China. The performance of WebShellkiller (2021) is essentially the same under the two different strategies, and the DR of this tool is only approximately 0.5, which is the lowest among all tools. SHELLPUB (2022) uses a multi-engine to detect malicious webshells with an accuracy rate of 0.887, recall rate of 0.699, and FPR of 0.009. WEBDIR+ (2021) employs different strategies to detect malicious webshells and has the lowest FPR among all tools. The DR of this tool is 0.874. This indicates that WEBDIR+ (2021) adopts a cautious strategy for detection. CloudWalker (2018) is based on abstract syntax trees and machine learning to detect malicious webshells, and its performance is second only to D shield (2023). The accuracy, DR, and FPR of this tool are 0.967, 0.923, and 0.008, respectively.

5. Conclusion

The opcode sequences of PHP-based malicious webshells are usually too long and contain many irrelevant features, which affect the performance of the detection method. To address the issue of excessively long opcode sequences of PHP-based malicious webshells, we proposed an abstract syntax tree simplification method called ASTSM. ASTSM adopts different simplification strategies for the three main types of nodes in the abstract syntax tree to compress the length and reduce the noise of malicious webshell opcode sequences. Subsequently, we propose a PHP-based malicious webshell detection model that uses an EDRN to capture the features of malicious webshell opcode sequences. Experimental results based on public datasets demonstrate that the EDRN can detect PHP-based malicious webshells more effectively than traditional RNNs. ASTSM can effectively enhance the performance of RNNs in detecting malicious webshells. The comparative experimental results prove that our method offers better detection performance than existing approaches, and it also outperforms existing mainstream tools, particularly when the training set is relatively small. When only 20% of the datasets are used for training, the accuracy, recall, and F1 scores of the proposed method were 0.985, 0.983, and 0.980, respectively. When 60% of the datasets were used for training, the metrics increased to 0.993, 0.991, and 0.990.

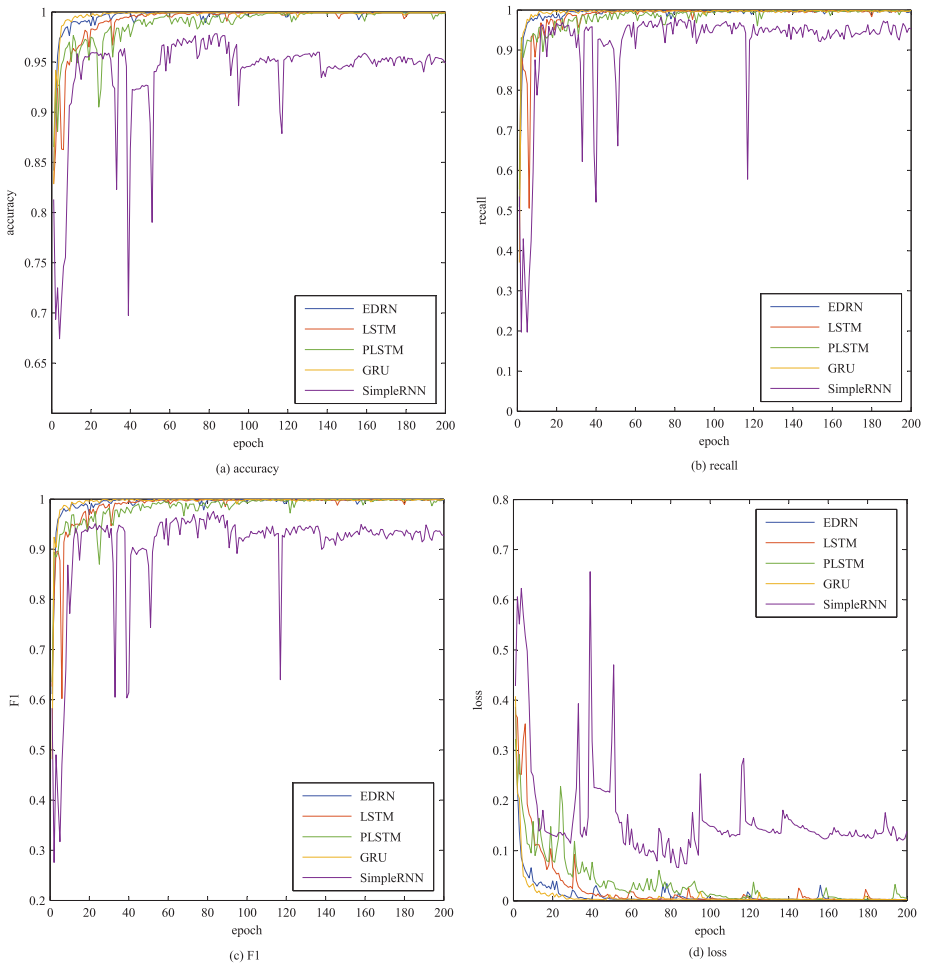


Fig. 14. Impact of epoch on model performance during training.

Table 10
Comparison results with existing mainstream tools.

Tools	Version	Accuracy	DR	F1	FPR	FNR
PHP Shell Detector (2016)	V1.1 by Python	0.809	0.910	0.771	0.247	0.090
D shield (2023)	V2.1.7.2	0.977	0.941	0.967	0.003	0.059
PHP Malware Finder (2022)	–	0.889	0.740	0.937	0.030	0.260
WebShellkiller (2021) (Strategy A)	V3.3.0.2	0.819	0.490	0.658	≈ 0	0.510
WebShellkiller (2021) (Strategy B)	V3.3.0.2	0.820	0.512	0.668	0.011	0.488
SHELLPUB (2022)	V1.8.3	0.887	0.699	0.815	0.009	0.301
WEBDIR+ (2021)	V2021-0628-1400	0.955	0.874	0.933	0	0.126
CloudWalker (2018)	V1.0	0.967	0.923	0.953	0.008	0.077
Our method	–	0.993	0.991	0.990	0.006	0.009

However, during our experiments, our method could not effectively distinguish between normal and malicious webshell samples with the same opcode sequence. In the future, we will attempt to analyze the relevant information of malicious webshells from multiple perspectives and combine other features, such as the statistical features of PHP scripts, to detect malicious webshells, with the aim of reducing the FPR of our method. This study adopted Word2vec to convert opcode sequences into word vectors. In future studies, we aim to explore the use of large language models to capture the semantic information of opcode sequences better.

CRediT authorship contribution statement

Bailin Xie: Writing – original draft, Methodology, Investigation, Conceptualization. **Qi Li:** Validation, Software, Formal analysis. **Yu Wang:** Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

This work was supported by the National Natural Science Foundation of China (Grant No. U21A20463), and the Science and Technology Project of Guangzhou (Grant No. 2024A03J0403).

chạy thực nghiệm EDRN vs nh model ≠ trên nh giá trị train : test : val ≠ nhau
Trên dataset 0 có ASTSM

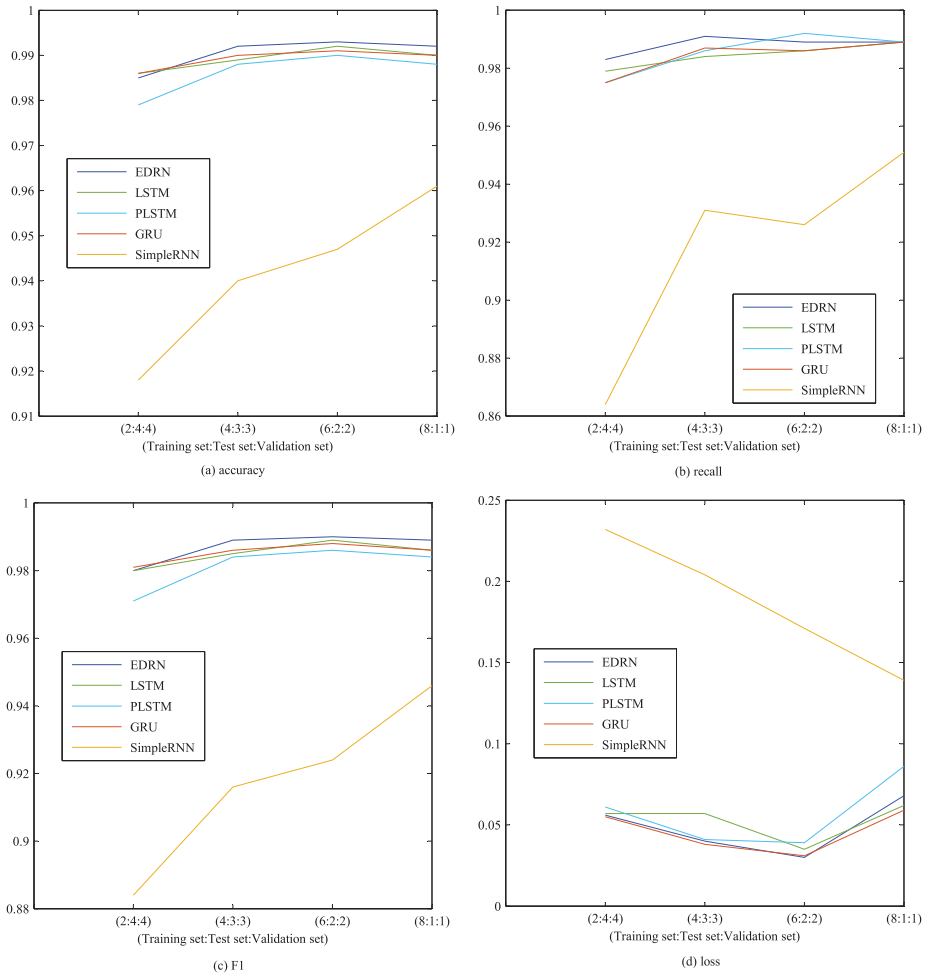


Fig. 15. Experimental results of EDRN and traditional RNNs based on datasets not processed by ASTSM.

Chạy thực nghiệm EDRN vs nh model ≠ trên nh giá trị train : test : val ≠ nhau
Trên dataset có ASTSM

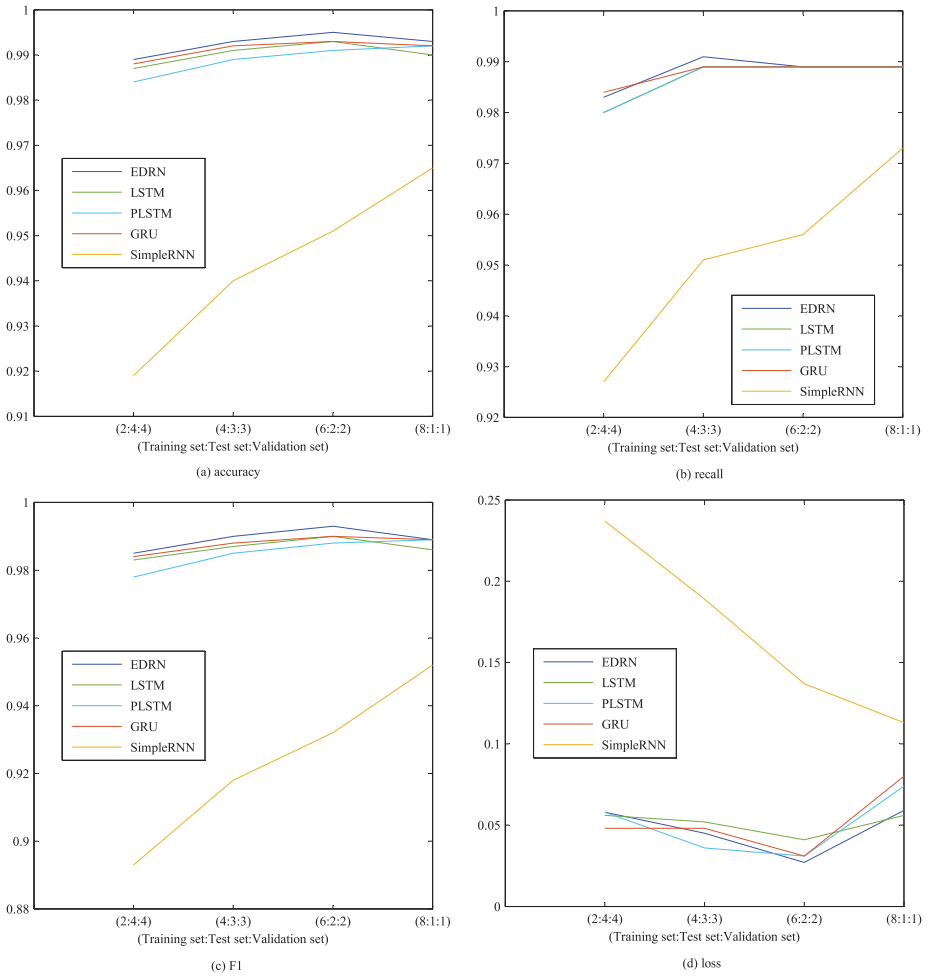


Fig. 16. Experimental results of EDRN and traditional RNNs based on datasets processed by ASTSM.

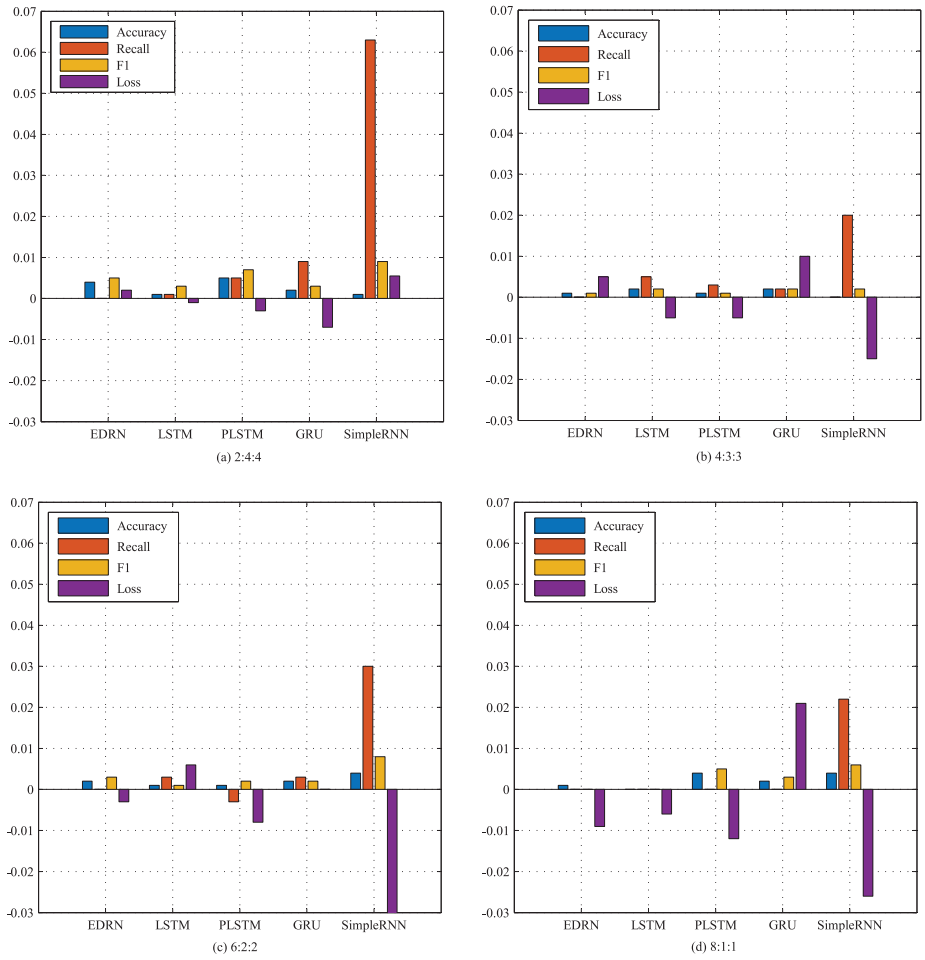


Fig. 17. Changes in the performance of RNNs after ASTSM is applied.

References

BlackArch, 2021. BlackArch. URL: https://github.com/BlackArch/webshells. January, 2023.

Cheng, B., Guo, Y., Ren, Y., Yang, G., Xu, G., 2022. MSDetector:A static PHP webshell detection system based on deep-learning. In: International Symposium on Theoretical Aspects of Software Engineering. pp. 155–172, https://springer.longhoe.net/chapter/10.1007/978-3-031-10363-6_11.

Cho, K., van Merrienboer, B., Gülçehre, Ç., Bougares, F., Schwenk, H., Bengio, Y., 2014. Learning phrase representations using RNN encoder-decoder for statistical machine translation. CoRR abs/1406.1078, arXiv:1406.1078.

CloudWalker, 2018. CloudWalker. URL: https://github.com/chaitin/cloudwalker. January, 2024.

Craftcms, 2022. Craftcms. URL: https://github.com/craftcms/cms. January, 2023.

Cui, H., Huang, D., Fang, Y., Liu, L., Huang, C., 2018. Webshell detection based on random forest-gradient boosting decision tree algorithm. In: 2018 IEEE Third International Conference on Data Science in Cyberspace. DSC, pp. 153–160, https://ieeexplore.ieee.org/abstract/document/8411851.

D shield, 2023. D shield. URL: http://www.d99net.net/. January, 2024.

Fang, Y., Qiu, Y., Liu, L., Huang, C., 2018. Detecting webshell based on random forest with fasttext. In: Proceedings of the 2018 International Conference on Computing and Artificial Intelligence. pp. 52–56, https://dl.acm.org/doi/abs/10.1145/3194452.3194470.

Feng, P., Wei, D., Li, Q., Wang, Q., Hu, Y., Xi, N., Ma, J., 2024. GlareShell: Graph learning-based PHP webshell detection for web server of industrial internet. Comput. Netw. 245, 110406. http://dx.doi.org/10.1016/j.comnet.2024.110406.

Hannousse, A., Nait-Hamoud, M.C., Yahiouche, S., 2023. A deep learner model for multi-language webshell detection. Int. J. Inf. Secur. 22 (1), 47–61. http://dx.doi.org/10.1007/s10207-022-00615-5.

Hannousse, A., Yahiouche, S., 2021. Handling webshell attacks:A systematic mapping and survey. Comput. Secur. 108, 102366. http://dx.doi.org/10.1016/j.cose.2021.102366.

Hochreiter, S., Schmidhuber, J., 1997. Long short-term memory. Neural Comput. 9 (8), 1735–1780. http://dx.doi.org/10.1162/neco.1997.9.8.1735.

JohnTroony, 2020. JohnTroony. URL: https://github.com/JohnTroony/php-webshells. January, 2023.

Li, Y., Huang, J., Ikusan, A., Mitchell, M., Zhang, J., 2019. Shellbreaker:Automatically detecting php-based malicious web shells. Comput. Secur. 87, 1–11. http://dx.doi.org/10.1016/j.cose.2019.101595.

Lipton, Z.C., Berkowitz, J., 2015. A critical review of recurrent neural networks for sequence learning. arXiv preprint arXiv:1506.00019, https://arxiv.org/pdf/1506.00019.

Liu, Z., Li, D., Wei, L., 2022. A new method for webshell detection based on bidirectional gru and attention mechanism. Secur. Commun. Netw. 2022, 1–11. http://dx.doi.org/10.1155/2022/3434920.

Nguyen, N.H., Le, V.H., Phung, V.O., Du, P.H., 2019. Toward a deep learning approach for detecting php webshell. In: Proceedings of the 10th International Symposium on Information and Communication Technology. pp. 514–521, https://dl.acm.org/doi/abs/10.1145/3368926.3369733.

Pan, Z., Chen, Y., Chen, Y., Shen, Y., Guo, X., 2021. Webshell detection based on executable data characteristics of php code. Wirel. Commun. Mob. Comput. 2021, 1–12. http://dx.doi.org/10.1155/2021/5533963.

Pang, B., Liang, G., Yang, J., Chen, Y., Wang, X., He, W., 2023. CWSOGG: Catching web shell obfuscation based on genetic algorithm and generative adversarial network. Comput. J. 66 (5), 1295–1309. http://dx.doi.org/10.1109/TITS.2021.3122979.

PHP Malware Finder, 2022. PHP malware finder. URL: https://github.com/nbs-system/php-malware-finder/. January, 2024.

PHP-Parser, 2024. PHP-parser. URL: https://github.com/nikic/PHP-Parser. January, 2024.

PHP Shell Detector, 2016. PHP shell detector. URL: https://github.com/emposha/Shell-Detector. January, 2023.

Sak, H., Senior, A.W., Beaufays, F., 2014. Long short-term memory based recurrent neural network architectures for large vocabulary speech recognition. CoRR abs/1402.1128, arXiv:1402.1128.

Shang, M., Han, X., Zhao, C., Cui, Z., Du, D., Jiang, B., 2024. Multi-language webshell detection based on abstract syntax tree and treelstm. In: Proceedings of the 27th International Conference on Computer Supported Cooperative Work in Design. pp. 377–382. http://dx.doi.org/10.1109/CSCWD61410.2024.10580271.

SHELLPUB, 2022. SHELLPUB. URL: https://www.shellpub.com/. January, 2024.

Suwais, K., Hnaif, A.A., Almanasra, S., 2023. An alternative static taint analysis framework to detect PHP web shell-based web attacks. Int. J. Adv. Soft Comput. Appl. 15 (3), 117–131, http://www.i-csrs.org/Volumes/ijasca/2023.3.8.pdf.

Tian, Y., Wang, J., Zhou, Z., Zhou, S., 2017. CNN-webshell:malicious web shell detection with convolutional neural network. In: Proceedings of the 2017 VI International Conference on Network,Communication and Computing. pp. 75–79, https://dl.acm.org/doi/abs/10.1145/3171592.3171593.

VLD, 2022. VLD. URL: https://pecl.php.net/package/vld. January, 2023.

V.Le, H., N.Nguyen, T., N.Nguyen, H., Le, L., 2023. An efficient hybrid webshell detection method for webserver of marine transportation systems. IEEE Trans. Intell. Transp. Syst. 24 (2), 2630–2642. http://dx.doi.org/10.1109/TITS.2021.3122979.

WEBDIR+, 2021. WEBDIR+. URL: https://scanner.baidu.com/. January, 2024.

WebShellkiller, 2021. WebShellkiller. URL: https://support.sangfor.com.cn/productTool/. January, 2024.

WordPress, 2022. WordPress. URL: https://github.com/WordPress/WordPress. January, 2023.

Xie, B., Li, Q., 2022. Webshell detection based on explicit duration recurrent network. In: Proceedings of the 13th International Symposium on Cyberspace Safety and Security. pp. 55–65, https://link.springer.com/chapter/10.1007/978-3-030-94029-4_4.

Xie, B., Li, Q., Kuang, J., Wei, N., Wang, Y., 2023. EDRN-based propagation model for popular microblog information detection. Inform. Sci. 640, 119075. http://dx.doi.org/10.1016/j.ins.2023.119075.

Xl7dev, 2017. Xl7dev. URL: https://github.com/xl7dev/WebShell. January, 2023.

Yii2, 2022. Yii2. URL: https://github.com/yiisoft/yii2. January, 2023.

Ysrc, 2018. Ysrc. URL: https://github.com/ysrc/webshell-sample. January, 2023.

Yu, S.Z., 2022. Explicit duration recurrent networks. IEEE Trans. Neural Netw. Learn. Syst. 33 (7), 3120–3130. http://dx.doi.org/10.1109/TNNLS.2021.3051019.

Zhang, H., Guan, H., Yan, H., Li, W., Yu, Y., Zhou, H., Zeng, X., 2018. Webshell traffic detection with character-level features based on deep learning. IEEE Access 6, 75268–75277. http://dx.doi.org/10.1109/ACCESS.2018.2882517.

Zhang, H., Liu, M., Yue, Z., Xue, Z., Shi, Y., He, X., 2020. A php and jsp web shell detection system with text processing based on machine learning. In: 2020 IEEE 19th International Conference on Trust,Security and Privacy in Computing and Communications. pp. 1584–1591, https://ieeexplore.ieee.org/abstract/document/9343056/.

Zhao, J., Lu, Y., Wang, X., Zhu, K., Yu, L., 2021. WTA:a static taint analysis framework for PHP webshell. Appl. Sci. 11 (16), 7763. http://dx.doi.org/10.3390/app11167763.

Zhou, A., Luktarhan, N., Ai, Z., 2021. Research on webshell detection method based on regularized neighborhood component analysis (RNCA). Symmetry 13 (7), 1202. http://dx.doi.org/10.3390/sym13071202.

Zhou, X., Zafarani, R., 2024. Research and application of artificial intelligence based webshell detection model: A literature review. arxiv preprint arxiv:2405.00066, https://arxiv.org/pdf/2405.00066.



Bailin Xie received his B.S. degree from Qingdao University, China. He received his M.S. degree and Ph.D. degree from Sun Yat-Sen University, China. He is currently an assistant professor at the school of Information Science and Technology in Guangdong University of Foreign Studies, China. His research interests focus on network security, social network and network behavior model algorithm.



Qi Li received her B.S. degree and M.S. degree from Guangdong University of Foreign Studies, China. She is currently a researcher at the Guangzhou Rural Commercial Bank, China. Her research interests focus on network security and social network.



Yu Wang received his B.S. degree and Ph.D. degree from Sun Yat-Sen University, China. He is currently an assistant professor at the institute of Artificial Intelligence in Guangzhou University, China. His research interests focus on network security.